Lecture 5: Priority Queues and Heaps

CSI2110 – Data Structures and Algorithms

Lecture study notes

Source document: `../lec05-priorityandheap.pdf`. These notes synthesize the source and add worked explanations. References use PDF page numbers. Pseudocode includes boundary cases.

Contents

1 Priority queues and ordering **2**

2 List implementations **2**

3 Heap shape and order **2**

4 Insertion and up-heap **3**

5 Removal and down-heap **3**

6 Heap construction **4**

6.1 Repeated insertion 4

6.2 Bottom-up heapify 4

6.3 Linear-time proof 5

6.4 Lecture exercise 5

7 Sorting through a priority queue **5**

8 In-place heapsort **5**

9 Study checks **6**

1 Priority queues and ordering

Source: PDF pages 4–13. A priority queue stores entries (k, v), where k is the priority key and v the associated value. Entries arrive in any order and leave according to priority. This lecture uses the minimum key as highest priority. Distinct entries may have equal keys.

- `insert(k,v)` adds an entry.
- `min()` inspects a minimum entry without removal.
- `removeMin()` removes and returns a minimum entry.
- `size()` and `isEmpty()` report container information. The lecture returns null on empty inspection/removal.

A comparator returns a negative result if $x < y$, zero for a tie, and positive if $x > y$. Reversing comparison changes the priority convention.

A total order requires comparability, antisymmetry, and transitivity. Numeric and lexicographic orders are examples. Compare pairs by first coordinates, then second coordinates to break ties. Compare strings by the first differing character; a proper prefix comes before its longer extension.

Clarification: ordering by string length alone ties distinct equal-length strings. This is a total preorder on strings (or total order on length-equivalence classes), not an antisymmetric order on distinct strings. It can still serve as a priority comparison. A lexicographic tie-breaker gives an order distinguishing equal-length strings. Tied priorities do not guarantee FIFO service unless explicitly implemented.

Java detail: use `Integer.compare(a,b)` rather than `a-b`, which can overflow for extreme integers.

2 List implementations

Source: PDF pages 15–20. An unsorted positional list inserts at an end in $O(1)$. Minimum lookup scans all entries in $\Theta(n)$; removing the known minimum position then costs $O(1)$. It still scans if the current data happen to be sorted because the implementation maintains no sorted invariant.

A sorted doubly linked positional list maintains increasing keys. Insertion may traverse n entries to find its spot, taking $O(n)$ worst case. Minimum inspection/removal use the first node in $O(1)$.

Implementation	Insert	Minimum	Remove minimum
Unsorted positional list	$O(1)$	$\Theta(n)$	$\Theta(n)$
Sorted positional list	$O(n)$	$O(1)$	$O(1)$
Binary min-heap	$O(\log n)$ repair	$O(1)$	$O(\log n)$

All store $O(n)$ entries. The sorted-list removal cost assumes linked positional removal; removing the front of a shifting array list would cost $O(n)$.

3 Heap shape and order

Source: PDF pages 22–26,53. A min-heap has two independent invariants: it is a complete binary tree, and every parent key is at most every child key. The minimum is at the root. Siblings and unrelated subtrees are not ordered relative to each other; the heap array is generally not sorted.

For $n \geq 1$, height is $\lfloor \log_2 n \rfloor$. A zero-based array stores entries at $0, \ldots, n-1$:

$$\text{left}(i) = 2i + 1, \quad \text{right}(i) = 2i + 2, \quad \text{parent}(i) = \lfloor (i-1)/2 \rfloor \quad (i > 0).$$

A child exists if its index is $< n$. Leaves occupy indices $\lfloor n/2 \rfloor, \ldots, n-1$; for $n \geq 2$, the last internal node is $\lfloor n/2 \rfloor - 1$.

With one-based indexing, children are $2i, 2i+1$ and parent is $\lfloor i/2 \rfloor$. Do not mix conventions. A max-heap instead satisfies parent key at least child key.

4 Insertion and up-heap

Source: PDF pages 34–39. Append to the next available array slot, preserving complete shape, then repair the ancestor path:

```
insert(k, v):
    append (k, v) to heap
    upHeap(heap.size - 1)

upHeap(j):
    while j > 0:
        p = floor((j - 1) / 2)
        if key(p) <= key(j): return
        swap heap[p] and heap[j]
        j = p
```

At most h swaps are needed, so repair is $O(\log n)$. Stop on equality because duplicate keys are valid.

Slide insertion trace:

```
Before: [3,4,7,21,10,20,8,22,28,13,19,25]
Append 6 at index 12; swap with 20 at 5, then 7 at 2.
After:  [3,4,6,21,10,7,8,22,28,13,19,25,20]
```

The new key stops below root 3.

An array resize can make one insertion $O(n)$. With doubling, resizing contributes amortized $O(1)$, giving amortized $O(\log n)$ heap insertion. If sufficient capacity is available, worst-case insertion is $O(\log n)$.

5 Removal and down-heap

Source: PDF pages 27–33. Save the root, move the last entry to the root, remove the last slot, and repair downward. Using the last slot preserves complete shape.

```
removeMin():
    if n == 0: return null
    result = heap[0]
    heap[0] = heap[n - 1]
    remove final slot; n = n - 1
    if n > 0: downHeap(0, n)
    return result

downHeap(j, n):  // active range [0, n)
    while 2*j + 1 < n:
        small = 2*j + 1
        right = 2*j + 2
        if right < n and key(right) < key(small):
            small = right
        if key(j) <= key(small): return
        swap heap[j] and heap[small]
        j = small
```

Key idea. Choose the smaller child in a min-heap. Choosing the larger can leave the parent greater than the unchosen smaller child.

Only the moved entry can violate order after a swap. It descends at most h levels, so removal is $O(\log n)$ without array shrinking. A last-level internal node may have only a left child; check whether the right child exists.

Slide removal trace (a separate starting heap):

```
Before: [3,4,6,21,10,7,20,22,28,13,19,25,20]
Move final 20 to root and remove final slot.
Repair follows indices 0 -> 1 -> 4 -> 9.
After:  [4,10,6,21,13,7,20,22,28,20,19,25]
```

The returned key is 3.

6 Heap construction

Source: PDF pages 41–59.

6.1 Repeated insertion

Insert all entries sequentially. The total is bounded by $\sum_{k=1}^{n} O(\log k) = O(n \log n)$, with worst-case $\Theta(n \log n)$ for adversarial order. Favorable orders can require fewer swaps.

6.2 Bottom-up heapify

Leaves are heaps already. Process internal nodes from last to first:

```
buildHeap(A):
    n = A.length
    for i = floor(n / 2) - 1 down to 0:
        downHeap(i, n)
```

Before each repair, both child subtrees are heaps. Down-heap makes the current subtree a heap. At index zero, the entire array becomes a heap.

6.3 Linear-time proof

Charging every repair $O(\log n)$ gives a loose $O(n \log n)$ bound. Most nodes are near leaves and can move only a few levels. At depth i, at most 2^i nodes can move at most $h - i$ levels:

$$\begin{aligned} \text{swaps} &\leq \sum_{i=0}^{h-1} (h-i)2^i = 2^h \sum_{j=1}^{h} \frac{j}{2^j} \\ &\leq 2^h \cdot 2 \leq 2n. \end{aligned}$$

Loop/comparison overhead is also $O(n)$, so total time is $\Theta(n)$. Iterative repair gives $O(1)$ auxiliary space.

6.4 Lecture exercise

```
Input: [14,9,8,25,5,11,27,16,15,4,12,6,7,23,20]
Process indices: 6, 5, 4, 3, 2, 1, 0
Final: [4,5,6,15,9,7,20,16,25,14,12,11,8,23,27]
```

Check that each parent is at most its children. Tied priorities can lead to different valid heap arrays.

7 Sorting through a priority queue

Source: PDF pages 61–64,82. Insert every input entry and repeatedly extract the minimum into the output. The comparator determines output order.

PQ structure	Build	Extraction	Worst-case total
Unsorted list	$O(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Sorted list	$O(n^2)$	$O(n)$	$\Theta(n^2)$
Heap, repeated inserts	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Heap, bottom-up build	$\Theta(n)$	$O(n \log n)$	$O(n \log n)$

Assume efficient input/output access. Repeated front removal from a shifting array list would itself cost $\Theta(n^2)$; scan the input array or heapify it directly. A separate heap/output implementation needs $O(n)$ extra storage.

8 In-place heapsort

Source: PDF pages 65–81. For ascending output, build a max-heap. Move its root (the current maximum) to the last active slot, shorten the active prefix, and repair that prefix. The suffix is sorted.

```
heapSort(A):
    n = A.length
    buildMaxHeap(A)
    for end = n - 1 down to 1:
        swap A[0] and A[end]
        downMaxHeap(0, end)  // active range [0, end)
```

Max-heap repair chooses the larger child and stops when the parent is at least that child. Max-heap construction uses the same bottom-up index order with reversed comparisons.

Invariant: before extraction, the active prefix is a max-heap, and the suffix contains the largest entries in ascending order. Repair must exclude the sorted suffix.

```
Initial max-heap: [8,5,6,1,4]
First repair:     [6,5,4,1 | 8]
Second repair:    [5,1,4 | 6,8]
Third repair:     [4,1 | 5,6,8]
Sorted:           [1,4,5,6,8]
```

Build is $\Theta(n)$, and all extraction work is $O(n \log n)$. Standard heapsort has $\Theta(n \log n)$ worst-case time and $O(1)$ auxiliary space with iterative repair. It is generally not stable: tied entries may change relative order. A min-heap placing extracted minima at the right end would produce descending output.

9 Study checks

1. Does a heap need to be full? No: completeness is the shape condition.
2. Is the heap array sorted? No: order constrains parents and children, not all pairs.
3. Why choose the last entry to replace the root? It preserves shape without shifting.
4. Why is bottom-up heapify linear? Most nodes have small subtree height.
5. Why shrink the active range in heapsort? Otherwise repairs disturb the sorted suffix.